

实验：工具链与工程

涵盖 2.1 / 2.2 实验任务 · 2.3 为选读内容

实验 2.1 交叉编译全流程

实验目标

- 确认 host 与 target 的架构和系统信息，完成三要素对照表
- 查询交叉编译器目标三元组与 sysroot 信息
- 使用 `chapters/ch02/code/hello/` 完成交叉编译，部署到 LicheePi 4A 并正确执行
- 对构建产物执行 `readelf -h`、`readelf -l`、`ldd`，摘录关键字段并判断链接方式

实验环境

项目	要求
主机环境	Linux，已安装 RISC-V Linux 交叉工具链
目标板	LicheePi 4A / RevyOS
软件依赖	<code>make</code> 、 <code>ssh</code> 、 <code>scp</code> 、 <code>file</code> 、 <code>readelf</code> 、 <code>ldd</code>
代码路径	<code>chapters/ch02/code/hello/</code>

任务 1：采集 host 与 target 信息

```
# 主机
$ uname -m; hostname
$ riscv64-unknown-linux-gnu-gcc -dumpmachine
$ riscv64-unknown-linux-gnu-gcc --version | head -n 1
$ riscv64-unknown-linux-gnu-gcc -print-sysroot

# 板端
$ uname -m; hostname; head -n 5 /etc/os-release
```

若 `-print-sysroot` 为空：

```
$ riscv64-unknown-linux-gnu-gcc -v -E - < /dev/null 2>&1 | head -n 30
```

填写 host / target / sysroot 三要素对照表，用三到五句话描述从编辑 C 源码到板端运行的完整路径。

任务 2：交叉编译与部署

```
$ cd chapters/ch02/code/hello
$ make clean && make
$ file hello          # 应显示 RISC-V ELF

$ scp hello user@board-ip:~/
$ ssh user@board-ip 'chmod +x ~/hello && file ~/hello && ./hello'
# 预期输出: Hello, RISC-V Linux!
```

可选验证：在主机执行 `./hello`，观察错误信息并截图——说明为何不能在 host 验证。

任务 3：ELF 分析

```
$ readelf -h hello      # 关注 Class、Machine、Type
$ readelf -l hello      # 观察 LOAD 段
$ readelf -S hello | head -n 15  # 浏览节头
$ ldd hello             # 动态库依赖（静态链接则注明）
```

```
# 可选：板端对比
```

```
$ ssh user@board-ip 'ldd ~/hello'
```

用表格归纳：Class、Machine、Type、是否动态链接、主要依赖库。

验收标准

验证项	预期现象
host 架构	与板端不同
target 架构	riscv64
三元组	含 linux-gnu
make	成功生成 hello
主机 file	RISC-V ELF
板端运行	Hello, RISC-V Linux!
readelf -h	Machine = RISC-V
ldd	列出依赖或注明静态链接

实验 2.2 Makefile 与工程目录规范

实验目标

- 读懂并演示 `chapters/ch02/code/hello/Makefile` 的增量构建与 `clean`
- 通过 `-Wall` 体验编译器警告与修复
- 基于 `chapters/ch02/code/project-template/` 完成多文件交叉编译
- 部署 `build/app` 到板端并运行，验证 `make clean` 可删除 `build/`

实验环境

项目	要求
单文件工程	<code>chapters/ch02/code/hello/</code>
多文件工程	<code>chapters/ch02/code/project-template/</code>
软件依赖	GNU <code>make</code> 、交叉 <code>gcc</code> 、 <code>scp</code> 、 <code>ssh</code>

任务 1：单文件 Makefile

```
$ cd chapters/ch02/code/hello
$ cat Makefile

# 完整构建流程
$ make clean && make
$ make                # 第二次应提示 up to date
$ touch main.c
$ make                # 应重新编译

# 清理
$ make clean
$ ls hello 2>&1        # hello 应不存在
$ make
```

警告实验：在 `main.c` 中临时引入未使用变量 (`int unused = 0;`)，执行 `make` 观察 `-Wall` 警告，修复后恢复原代码。

任务 2：命令行覆盖变量

```
$ make clean && make CFLAGS='-Wall -Wextra -O0 -g'
$ file hello
```

说明 Makefile 变量可被命令行覆盖，便于临时调试。

任务 3：多文件工程

```
$ cd chapters/ch02/code/project-template
$ find . -type f | sort

$ make clean && make
$ file build/app          # RISC-V ELF

# 部署到板端
$ scp build/app user@board-ip:~/
$ ssh user@board-ip 'chmod +x ~/app && ./app'
# 预期输出: Hello, RISC-V Linux!

# 清理重构
$ make clean
$ ls build 2>&1           # build/ 应不存在
$ make                  # 一键重建
```

在实验报告中用示意图或树状列表说明 `src/`、`include/`、`build/` 各自职责。用三五句话解释规则 `hello: main.c` 中三行的含义。

验收标准

验证项

预期现象

首次 build

成功

二次 make	无重编或提示 up to date
<code>touch</code> 后	触发重编
<code>clean</code>	<code>hello</code> 不存在
警告	能触发并消除
多文件 make	成功生成 <code>build/app</code>
板端 <code>./app</code>	<code>Hello, RISC-V Linux!</code>
清理重构	<code>build/</code> 消失后一键重建